

Partial Order Reduction and Abstraction

Csanád Telbisz

September 24, 2026



Sources of complexity in concurrent software verification

Data complexity

The state space grows exponentially with the number of variables and the size of their domains.

⇒ **Abstraction**

Thread interleavings

The number of executions grows combinatorially with the number of processes and actions.

⇒ **Partial order reduction (POR)**

Sources of complexity in concurrent software verification

Data complexity

The state space grows exponentially with the number of variables and the size of their domains.

⇒ **Abstraction**

Thread interleavings

The number of executions grows combinatorially with the number of processes and actions.

⇒ **Partial order reduction (POR)**

- ▶ Both are well established — *in isolation*
- ▶ Few works combine them, and those that do are tied to a particular POR algorithm or a particular abstract domain

Sources of complexity in concurrent software verification

Data complexity

The state space grows exponentially with the number of variables and the size of their domains.

⇒ **Abstraction**

Thread interleavings

The number of executions grows combinatorially with the number of processes and actions.

⇒ **Partial order reduction (POR)**

- ▶ Both are well established — *in isolation*
- ▶ Few works combine them, and those that do are tied to a particular POR algorithm or a particular abstract domain

This talk

A **general** abstraction-aware POR approach: independent of the underlying POR algorithm and of the abstract domain.

Roadmap

1. Abstraction
2. Partial order reduction — Mazurkiewicz traces, dependency, source sets
3. Why the combination is *not* trivial
4. Abstraction-aware extension of POR
5. Abstract POR Correctness conditions
6. Evaluation

Take-away, up front

Abstraction-aware POR is **as good as the abstraction**: the more details the abstraction ignores, the better the reduction.

Abstraction

Definition (Abstract domain)

A tuple $(\mathcal{D}, \top, \perp, \preceq, \text{expr})$, where $\text{expr} : \mathcal{D} \mapsto \text{FOL}$ maps an abstract data state to a formula describing its meaning.

- ▶ A **concrete state** c : CFA locations of all processes (denoted $c(p)$ for each process p), plus a variable valuation ($c(x)$ for each variable x)
- ▶ An **abstract state** s : CFA locations of all processes tracked explicitly ($s(p)$), plus an abstract data state $d \in \mathcal{D}$
- ▶ $c \models s$ (s **represents** c) iff $\forall$ process p : $c(p) = s(p)$ and $\text{expr}(c) \Rightarrow \text{expr}(s)$ where $\text{expr}(c) = \bigwedge_{x \in \text{vars}} x = c(x)$, and $\text{expr}(s) = \text{expr}(d)$ for the data state d of s

Abstraction

Definition (Abstract domain)

A tuple $(\mathcal{D}, \top, \perp, \preceq, \text{expr})$, where $\text{expr} : \mathcal{D} \mapsto \text{FOL}$ maps an abstract data state to a formula describing its meaning.

- ▶ A **concrete state** c : CFA locations of all processes (denoted $c(p)$ for each process p), plus a variable valuation ($c(x)$ for each variable x)
- ▶ An **abstract state** s : CFA locations of all processes tracked explicitly ($s(p)$), plus an abstract data state $d \in \mathcal{D}$
- ▶ $c \models s$ (s **represents** c) iff $\forall$ process p : $c(p) = s(p)$ and $\text{expr}(c) \Rightarrow \text{expr}(s)$ where $\text{expr}(c) = \bigwedge_{x \in \text{vars}} x = c(x)$, and $\text{expr}(s) = \text{expr}(d)$ for the data state d of s
- ▶ A **precision** Π says what the abstraction tracks; $\text{vars}(\Pi)$ is the set of variables it has information about — $\text{expr}(s)$ only uses variables in $\text{vars}(\Pi)$

Abstraction

Definition (Abstract domain)

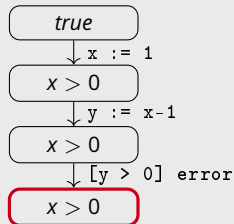
A tuple $(\mathcal{D}, \top, \perp, \preceq, \text{expr})$, where $\text{expr} : \mathcal{D} \mapsto \text{FOL}$ maps an abstract data state to a formula describing its meaning.

- ▶ A **concrete state** c : CFA locations of all processes (denoted $c(p)$ for each process p), plus a variable valuation ($c(x)$ for each variable x)
- ▶ An **abstract state** s : CFA locations of all processes tracked explicitly ($s(p)$), plus an abstract data state $d \in \mathcal{D}$
- ▶ $c \models s$ (s **represents** c) iff $\forall$ process p : $c(p) = s(p)$ and $\text{expr}(c) \Rightarrow \text{expr}(s)$ where $\text{expr}(c) = \bigwedge_{x \in \text{vars}} x = c(x)$, and $\text{expr}(s) = \text{expr}(d)$ for the data state d of s
- ▶ A **precision** Π says what the abstraction tracks; $\text{vars}(\Pi)$ is the set of variables it has information about — $\text{expr}(s)$ only uses variables in $\text{vars}(\Pi)$
- ▶ An abstract trace $w = \alpha_1 \dots \alpha_k$ from s_0 is **feasible** if it has a concretization $c_0 \xrightarrow{\alpha_1} \dots \xrightarrow{\alpha_k} c_k$ with $c_i \models s_i$; otherwise it is **spurious**

Abstract domain example: predicate abstraction

- ▶ Abstract data state: Boolean combination of predicates
- ▶ $\top = \text{true}$, $\perp = \text{false}$, $\preceq$ is implication
- ▶ Precision Π : a set of FOL predicates;
 $\text{vars}(\Pi)$: the variables occurring in them

$\Pi = \{x > 0\}$, so $\text{vars}(\Pi) = \{x\}$



- ▶ An abstract *error* trace, but **spurious**:
 $y = 0$ in every concretization
- ▶ If using CEGAR: the refiner detects this and refines Π

Partial order reduction: the problem

Running example

Process p_1

$\alpha_1: x := 1$

$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$

Partial order reduction: the problem

Running example

Process p_1

$\alpha_1: x := 1$

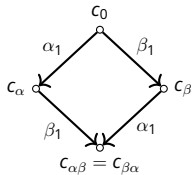
$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$

- ▶ Many interleavings lead to the **same outcome**



Partial order reduction: the problem

Running example

Process p_1

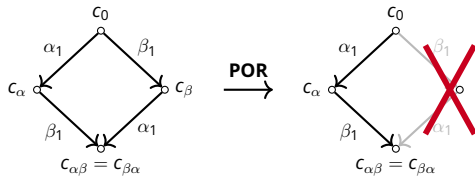
$\alpha_1: x := 1$

$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$



- ▶ Many interleavings lead to the **same outcome**
- ▶ **Key idea of POR:** define an equivalence relation on traces and explore a single representative (or as few as possible) of each equivalence class

Partial order reduction: the problem

Running example

Process p_1

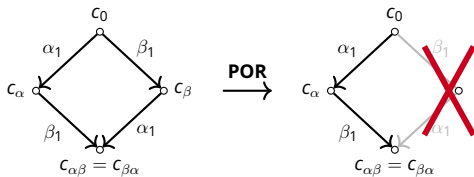
$\alpha_1: x := 1$

$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$



- ▶ Many interleavings lead to the **same outcome**
- ▶ **Key idea of POR:** define an equivalence relation on traces and explore a single representative (or as few as possible) of each equivalence class

Requirement of this equivalence

If a trace reaches an error state, then *all* traces in its equivalence class should reach an error state.

Mazurkiewicz traces

Running example

Process p_1

$\alpha_1: x := 1$

$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$

$\alpha_1.\alpha_2.\beta_1.\beta_2$

$\alpha_1.\beta_1.\alpha_2.\beta_2$

$\alpha_1.\beta_1.\beta_2.\alpha_2$

$\beta_1.\alpha_1.\alpha_2.\beta_2$

$\beta_1.\alpha_1.\beta_2.\alpha_2$

$\beta_1.\beta_2.\alpha_1.\alpha_2$

- Only α_1 and β_2 **interact** (both write x)

Mazurkiewicz traces

Running example

Process p_1

$\alpha_1: x := 1$

$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$

$\alpha_1.\alpha_2.\beta_1.\beta_2$

$\alpha_1.\beta_1.\alpha_2.\beta_2$

$\alpha_1.\beta_1.\beta_2.\alpha_2$

$\beta_1.\alpha_1.\alpha_2.\beta_2$

$\beta_1.\alpha_1.\beta_2.\alpha_2$

$\beta_1.\beta_2.\alpha_1.\alpha_2$

Class A:
 α_1 before β_2

Class B:
 β_2 before α_1

- ▶ Only α_1 and β_2 **interact** (both write x)
- ▶ The class is determined by their relative order

Mazurkiewicz traces

Running example

Process p_1

$\alpha_1: x := 1$

$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$

$\alpha_1.\alpha_2.\beta_1.\beta_2$

$\alpha_1.\beta_1.\alpha_2.\beta_2$

$\alpha_1.\beta_1.\beta_2.\alpha_2$

$\beta_1.\alpha_1.\alpha_2.\beta_2$

$\beta_1.\alpha_1.\beta_2.\alpha_2$

$\beta_1.\beta_2.\alpha_1.\alpha_2$

Class A:
 α_1 before β_2

Class B:
 β_2 before α_1

- ▶ Only α_1 and β_2 **interact** (both write x)
- ▶ The class is determined by their relative order

Mazurkiewicz trace = an equivalence class

Two traces are equivalent (denoted as $w_1 \approx w_2$) if they can be obtained from each other by successively swapping adjacent **independent** actions.

Dependency

Definition (Dependency relation)

Let A be the set of actions, and $D \subseteq A \times A$ be a binary, reflexive, symmetric relation. D is a dependency relation if $(\alpha, \beta) \notin D$ (α and β are **independent**) implies, for all states s :

- ▶ if $\alpha \in \text{enabled}(s)$, then $\beta \in \text{enabled}(s)$ iff $\beta \in \text{enabled}(\alpha(s))$ (no enabling/disabling)
- ▶ if $\alpha, \beta \in \text{enabled}(s)$, then $\beta(\alpha(s)) = \alpha(\beta(s))$ (commutativity)
- ▶ $\alpha(s)$ is the successor of s with respect to α — POR algorithms usually require an **action-deterministic state space** $\rightarrow$ the transfer relation must be a function

Dependency

Definition (Dependency relation)

Let A be the set of actions, and $D \subseteq A \times A$ be a binary, reflexive, symmetric relation. D is a dependency relation if $(\alpha, \beta) \notin D$ (α and β are **independent**) implies, for all states s :

- ▶ if $\alpha \in \text{enabled}(s)$, then $\beta \in \text{enabled}(s)$ iff $\beta \in \text{enabled}(\alpha(s))$ (no enabling/disabling)
- ▶ if $\alpha, \beta \in \text{enabled}(s)$, then $\beta(\alpha(s)) = \alpha(\beta(s))$ (commutativity)
- ▶ $\alpha(s)$ is the successor of s with respect to α — POR algorithms usually require an **action-deterministic state space** $\rightarrow$ the transfer relation must be a function
- ▶ Not usable directly: to decide whether two actions commute we would have to **explore α and β in both orders** — exactly what POR wants to avoid

Dependency

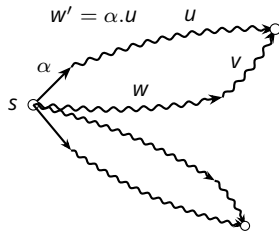
Definition (Dependency relation)

Let A be the set of actions, and $D \subseteq A \times A$ be a binary, reflexive, symmetric relation. D is a dependency relation if $(\alpha, \beta) \notin D$ (α and β are **independent**) implies, for all states s :

- ▶ if $\alpha \in \text{enabled}(s)$, then $\beta \in \text{enabled}(s)$ iff $\beta \in \text{enabled}(\alpha(s))$ (no enabling/disabling)
- ▶ if $\alpha, \beta \in \text{enabled}(s)$, then $\beta(\alpha(s)) = \alpha(\beta(s))$ (commutativity)
- ▶ $\alpha(s)$ is the successor of s with respect to α — POR algorithms usually require an **action-deterministic state space** $\rightarrow$ the transfer relation must be a function
- ▶ Not usable directly: to decide whether two actions commute we would have to **explore α and β in both orders** — exactly what POR wants to avoid
- ▶ In practice: a **syntactic over-approximation** D_S . $(\alpha, \beta) \in D_S$ iff
 - ▶ α and β are actions of the same process, or
 - ▶ $\text{vars}(\alpha) \cap \text{vars}(\beta) \neq \emptyset$, and at least one variable in $\text{vars}(\alpha) \cap \text{vars}(\beta)$ is written

How much do we have to explore? Source sets

For every trace w from s we want an explored trace w' that is equivalent to w or to an **extension** of w : i.e. $w.v \approx w'$ for some v .



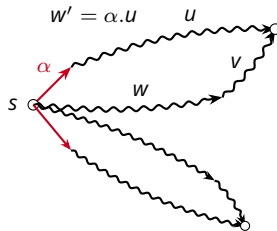
How much do we have to explore? Source sets

For every trace w from s we want an explored trace w' that is equivalent to w or to an **extension** of w : i.e. $w.v \approx w'$ for some v .

Weak initials & Source sets

$\alpha \in WI_s(w)$ iff there are u, v with $\alpha.u \approx w.v$.

S is a **source set** for a set W of traces in s if $WI_s(w) \cap S \neq \emptyset$ for each $w \in W$.



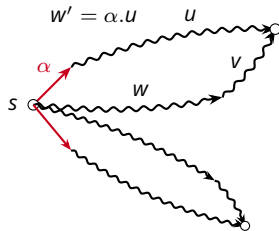
How much do we have to explore? Source sets

For every trace w from s we want an explored trace w' that is equivalent to w or to an **extension** of w : i.e. $w.v \approx w'$ for some v .

Weak initials & Source sets

$\alpha \in WI_S(w)$ iff there are u, v with $\alpha.u \approx w.v$.

S is a **source set** for a set W of traces in s if $WI_S(w) \cap S \neq \emptyset$ for each $w \in W$.



Theorem (Optimal DPOR paper, Abdulla et al. 2014)

It is **necessary** and **sufficient*** to explore the actions of a source set in each state to explore at least one trace from each Mazurkiewicz.

Source sets on the running example

Running example

Process p_1

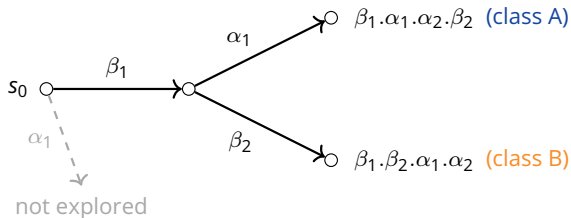
α_1 : $x := 1$

α_2 : $y := y+1$

Process p_2

β_1 : $z := z+1$

β_2 : $x := 2$



Source sets on the running example

Running example

Process p_1

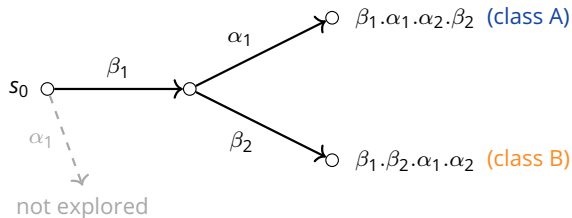
α_1 : $x := 1$

α_2 : $y := y+1$

Process p_2

β_1 : $z := z+1$

β_2 : $x := 2$



- β_1 is independent of both α_1 and α_2 , so it can be swapped to the front of every trace $\Rightarrow \beta_1 \in WI_{s_0}(w)$ for all w $\Rightarrow \{\beta_1\}$ is a source set in s_0

Source sets on the running example

Running example

Process p_1

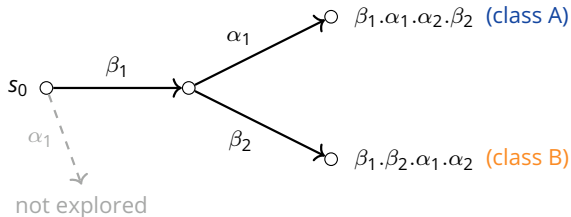
α_1 : $x := 1$

α_2 : $y := y+1$

Process p_2

β_1 : $z := z+1$

β_2 : $x := 2$



- ▶ β_1 is independent of both α_1 and α_2 , so it can be swapped to the front of every trace $\Rightarrow \beta_1 \in WI_{s_0}(w)$ for all w $\Rightarrow \{\beta_1\}$ is a source set in s_0
- ▶ No trace starting with α_1 can be swapped to obtain the class B trace, so $\alpha_1 \notin WI_{s_0}(w)$ for traces w of class B $\Rightarrow \{\alpha_1\}$ is not

Source sets on the running example

Running example

Process p_1

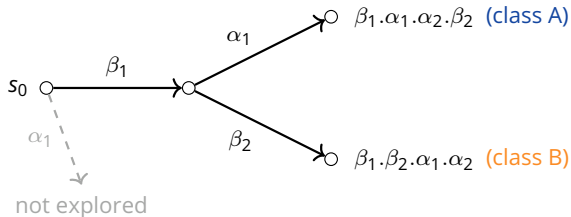
α_1 : $x := 1$

α_2 : $y := y+1$

Process p_2

β_1 : $z := z+1$

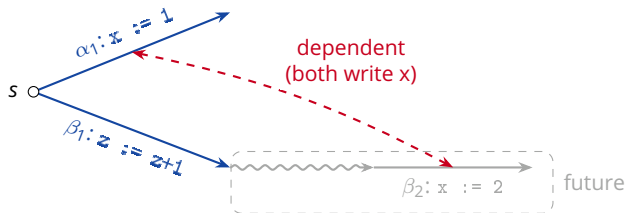
β_2 : $x := 2$



- ▶ β_1 is independent of both α_1 and α_2 , so it can be swapped to the front of every trace $\Rightarrow \beta_1 \in WI_{s_0}(w)$ for all $w \Rightarrow \{\beta_1\}$ is a source set in s_0
- ▶ No trace starting with α_1 can be swapped to obtain the class B trace, so $\alpha_1 \notin WI_{s_0}(w)$ for traces w of class B $\Rightarrow \{\alpha_1\}$ is not
- ▶ In s_1 , α_1 and β_2 are dependent $\Rightarrow$ both must be explored

Static POR

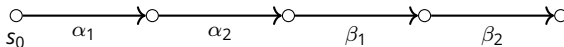
- ▶ **Look ahead** in each thread's control flow (without state space exploration): collect potential conflicts of currently enabled actions with possible **future actions** of other threads
- ▶ This is an over-approximation: the future action might not be executable later



$\Rightarrow$ If α_1 is explored from s , then β_1 must be explored too (the only way to explore β_2 before exploring α_1)

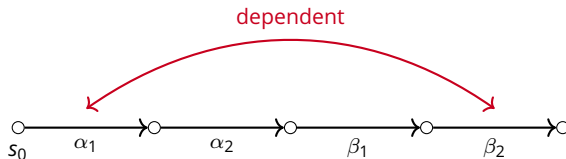
Dynamic POR

- ▶ Explore one trace depth-first, then look at the **search stack** for dependent actions that could be explored in the reversed order
- ▶ Insert **backtracking points** where another action has to be explored



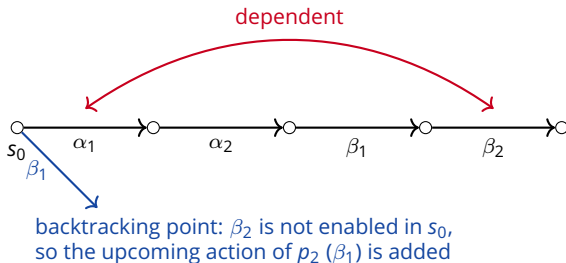
Dynamic POR

- ▶ Explore one trace depth-first, then look at the **search stack** for dependent actions that could be explored in the reversed order
- ▶ Insert **backtracking points** where another action has to be explored



Dynamic POR

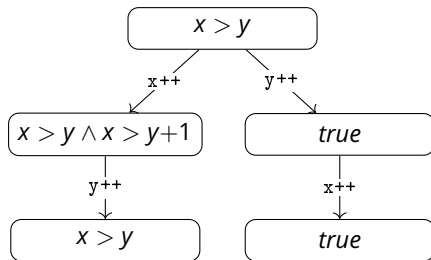
- ▶ Explore one trace depth-first, then look at the **search stack** for dependent actions that could be explored in the reversed order
- ▶ Insert **backtracking points** where another action has to be explored



Challenge: syntactic independence does not imply commutativity

- ▶ Predicate abstraction with $\Pi = \{x > y, x > y + 1\}$
- ▶ $x++$ and $y++$ have **no common variable**
 $\Rightarrow$ independent in D_S
- ▶ Yet the two orders reach **differently labelled** abstract states

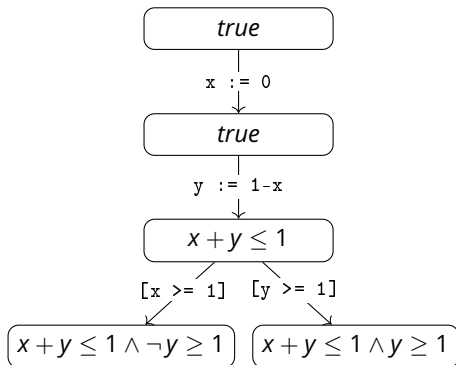
D_S is *not* a valid dependency relation in the abstract state space.



Challenge: syntactic independence does not imply no disabling

- ▶ Predicate abstraction with $\Pi = \{x + y \leq 1, y \geq 1\}$
- ▶ $[x \geq 1]$ and $[y \geq 1]$ have **no common variable**
 $\Rightarrow$ independent in D_S
- ▶ Yet the two actions **may disable each other** in the abstract state space

D_S is *not* a valid dependency relation in the abstract state space.



Abstraction-aware dependency relation

Let's complicate things further: we want to use a relaxed dependency relation.

Definition

Given an abstract state space with precision Π , $(\alpha, \beta) \in D_{\Pi}$ iff

- ▶ α and β are actions of the same process, *or*
- ▶ $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi) \neq \emptyset$, and at least one variable in $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi)$ is written by α or β

Abstraction-aware dependency relation

Let's complicate things further: we want to use a relaxed dependency relation.

Definition

Given an abstract state space with precision Π , $(\alpha, \beta) \in D_\Pi$ iff

- ▶ α and β are actions of the same process, *or*
- ▶ $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi) \neq \emptyset$, and at least one variable in $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi)$ is written by α or β
- ▶ Intuition: if the **source of the dependency is ignored** by the abstraction, there is no point in exploring both orderings of the two actions
- ▶ Still computed **syntactically**

Abstraction-aware dependency relation

Let's complicate things further: we want to use a relaxed dependency relation.

Definition

Given an abstract state space with precision Π , $(\alpha, \beta) \in D_\Pi$ iff

- ▶ α and β are actions of the same process, *or*
- ▶ $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi) \neq \emptyset$, and at least one variable in $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi)$ is written by α or β
- ▶ Intuition: if the **source of the dependency is ignored** by the abstraction, there is no point in exploring both orderings of the two actions
- ▶ Still computed **syntactically**
- ▶ Note that $D_\Pi \subseteq D_S$ for any precision Π

Abstraction-aware dependency relation

Let's complicate things further: we want to use a relaxed dependency relation.

Definition

Given an abstract state space with precision Π , $(\alpha, \beta) \in D_\Pi$ iff

- ▶ α and β are actions of the same process, or
- ▶ $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi) \neq \emptyset$, and at least one variable in $\text{vars}(\alpha) \cap \text{vars}(\beta) \cap \text{vars}(\Pi)$ is written by α or β
- ▶ Intuition: if the **source of the dependency is ignored** by the abstraction, there is no point in exploring both orderings of the two actions
- ▶ Still computed **syntactically**
- ▶ Note that $D_\Pi \subseteq D_S$ for any precision Π

D_S is already *not* a valid dependency relation in the abstract state space — and D_Π is an even smaller relation. Soundness is far from obvious.

Abstraction-aware dependency relation: example

$x := 1$ and $x := 2$

Dependent in D_S : both write x

- ▶ $\Pi_1 = \{x < y, z = 1\}$:
dependent in D_{Π_1}
- ▶ $\Pi_2 = \{0 < y, z = 1\}$:
independent in D_{Π_2} — the value of x is completely ignored

Back to the running example

Process p_1

$\alpha_1: x := 1$

$\alpha_2: y := y+1$

Process p_2

$\beta_1: z := z+1$

$\beta_2: x := 2$

If $x \notin \text{vars}(\Pi)$, then $(\alpha_1, \beta_2) \notin D_{\Pi}$.

$\Rightarrow$ every enabled action forms a source set on its own

$\Rightarrow$ 1 trace needs to be explored instead of 6

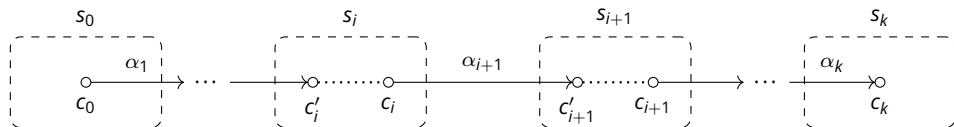
Partial feasibility

Definition

V is the set of variables. An abstract trace $w = \alpha_1 \dots \alpha_k$ is **partially feasible** from s_0 for $P \subseteq V$ if there are concrete states $c_0, \dots, c_k$ such that for each $0 \leq i \leq k$

- ▶ $c_i \models s_i$, and
- ▶ there is a c'_{i+1} with $c_i \xrightarrow{\alpha_{i+1}} c'_{i+1}$, $c'_{i+1} \models s_{i+1}$, $c'_{i+1}(p) = c_{i+1}(p)$ for each process p , and $c'_{i+1}(v) = c_{i+1}(v)$ for each variable $v \in P$.

$c_0, \dots, c_k$ is a **partial concretization** of w for P .



Partial feasibility: why it is the right notion

- ▶ Partial feasibility for $P = V$ is feasibility
- ▶ Feasible $\Rightarrow$ partially feasible for any P
- ▶ We use $P = \text{vars}(\Pi)$: only the variables the abstraction knows about can spoil it

Partial feasibility: why it is the right notion

- ▶ Partial feasibility for $P = V$ is feasibility
- ▶ Feasible $\Rightarrow$ partially feasible for any P
- ▶ We use $P = \text{vars}(\Pi)$: only the variables the abstraction knows about can spoil it

Partial feasibility motivation

Two independent actions, variables start at 0.

α : $[x=0]$ reach error

β : $[y=1]$ $y:=0$

Track only x . A $w = \alpha$ is a feasible error trace. However, β is also enabled in the abstract initial state: POR may explore only $w' = \beta.\alpha$ being equivalent to an extension of w .

w is **feasible** but w' is **not feasible** — however both are **partially feasible** for $\{x\}$, and both reach the error.

Abstract Mazurkiewicz traces

Theorem

Let $w_1 \approx^\Pi w_2$. Then w_1 is a partially feasible abstract trace from s for $\text{vars}(\Pi)$ *iff* w_2 is.

- ▶ $\approx^\Pi$: obtainable from each other by successively swapping adjacent actions independent in D_Π
- ▶ Simply put, **independent swaps** under D_Π **preserve partial feasibility**

Abstract Mazurkiewicz traces

Theorem

Let $w_1 \approx^\Pi w_2$. Then w_1 is a partially feasible abstract trace from s for $\text{vars}(\Pi)$ *iff* w_2 is.

- ▶ $\approx^\Pi$: obtainable from each other by successively swapping adjacent actions independent in D_Π
- ▶ Simply put, **independent swaps** under D_Π **preserve partial feasibility**
- ▶ Partial feasibility is a property of the **whole equivalence class**
- ▶ Proof idea: for a swap $q.\alpha.\beta.r \rightarrow q.\beta.\alpha.r$, α and β share no *tracked* written variable, so a partial concretization of $q.\alpha.\beta.r$ rearranges into one of $q.\beta.\alpha.r$

Classifying abstract Mazurkiewicz traces

Contains feasible trace	✓ ✓ ✓ ✓	✓ ✓ ✓ ✓	✗ ✗ ✗ ✗	✗ ✗ ✗ ✗
Contains infeasible trace	✗ ✗ ✗ ✗	✓ ✓ ✓ ✓	✓ ✓ ✓ ✓	✗ ✗ ✗ ✗
Contains partially feasible trace	✓ ✗ ✓ ✗	✓ ✗ ✓ ✗	✓ ✗ ✓ ✗	✓ ✗ ✓ ✗
Contains partially infeasible trace	✗ ✓ ✓ ✗	✗ ✓ ✓ ✗	✗ ✓ ✓ ✗	✗ ✓ ✓ ✗

$2^4 = 16$ combinations to start with.

Classifying abstract Mazurkiewicz traces

Contains feasible trace	✓	✓	✓	✓	✗	✗	✗	✗
Contains infeasible trace	✗	✗	✓	✓	✓	✓	✗	✗
Partially feasible for $vars(\Pi)$	✓	✗	✓	✗	✓	✗	✓	✗

All traces of a class are partially feasible, or none of them is (Theorem)

Classifying abstract Mazurkiewicz traces

Contains feasible trace



Contains infeasible trace



Partially feasible for $vars(\Pi)$



Impossible

A feasible trace is partially feasible for *any* set of variables.

Empty

Neither feasible nor infeasible trace
 $\Rightarrow$ the class is empty.

Classifying abstract Mazurkiewicz traces

Contains feasible trace	✓	✓	✗	✗
Contains infeasible trace	✗	✓	✓	✓
Partially feasible for $vars(\Pi)$	✓	✓	✓	✗
Example				

Examples assume $vars(\Pi) = \{x\}$ and all variables initially 0; **feasible** / **infeasible** traces.

Classifying abstract Mazurkiewicz traces

Contains feasible trace	✓	✓	✗	✗
Contains infeasible trace	✗	✓	✓	✓
Partially feasible for $vars(\Pi)$	✓	✓	✓	✗
Example				

Examples assume $vars(\Pi) = \{x\}$ and all variables initially 0; **feasible** / **infeasible** traces.

- ▶ Classes containing no feasible traces are **not interesting** — such traces may be pruned, no need to explore an equivalent at all
- ▶ We **cannot decide feasibility** during abstract state space exploration, only partial feasibility for $vars(\Pi)$

Classifying abstract Mazurkiewicz traces

Contains feasible trace	✓	✓	✗	✗
Contains infeasible trace	✗	✓	✓	✓
Partially feasible for $vars(\Pi)$	✓	✓	✓	✗
Example				

Examples assume $vars(\Pi) = \{x\}$ and all variables initially 0; **feasible** / **infeasible** traces.

Abstract POR correctness criterion

A POR algorithm needs to explore at least one representative of every **partially feasible** class.

Good news: involved reasoning but simple implementation

- ▶ Just replace $\approx$ by $\approx^\Pi$ in the definition of weak initials and it works

Definition (Weak initials, abstraction-aware)

For a trace w that is **partially feasible from s for $\text{vars}(\Pi)$** : $\alpha \in WI_s^\Pi(w)$ iff there are u, v with $\alpha.u \approx^\Pi w.v$ and **$w.v$ is partially feasible from s for $\text{vars}(\Pi)$** .

Good news: involved reasoning but simple implementation

- ▶ Just replace $\approx$ by $\approx^\Pi$ in the definition of weak initials and it works

Definition (Weak initials, abstraction-aware)

For a trace w that is **partially feasible from s for $\text{vars}(\Pi)$** : $\alpha \in WI_s^\Pi(w)$ iff there are u, v with $\alpha.u \approx^\Pi w.v$ and **$w.v$ is partially feasible from s for $\text{vars}(\Pi)$** .

- ▶ **In practice:** take any POR algorithm and use D_Π instead of D_S

Good news: involved reasoning but simple implementation

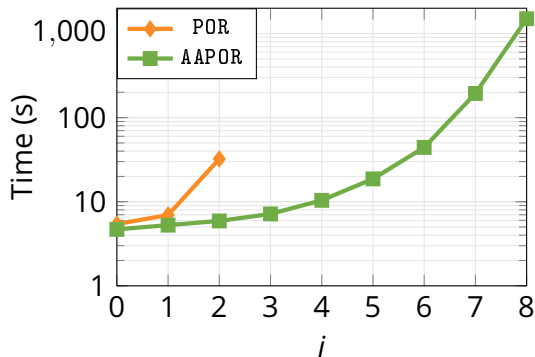
- ▶ Just replace $\approx$ by $\approx^\Pi$ in the definition of weak initials and it works

Definition (Weak initials, abstraction-aware)

For a trace w that is **partially feasible from s for $\text{vars}(\Pi)$** : $\alpha \in WI_s^\Pi(w)$ iff there are u, v with $\alpha.u \approx^\Pi w.v$ and **$w.v$ is partially feasible from s for $\text{vars}(\Pi)$** .

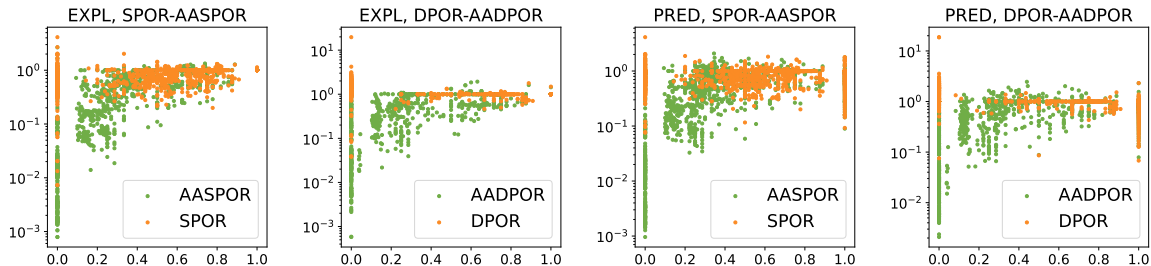
- ▶ **In practice**: take any POR algorithm and use D_Π instead of D_S
- ▶ On the theory side, quite some reasoning is needed to prove this correct, but the implementation is that simple

Experiments: exponential gain is possible



Synthetic benchmark with $\mathcal{O}(2^i)$ threads

Experiments: how much actual reduction?



- ▶ One point per CEGAR iteration on SV-COMP concurrency benchmarks
 - ▶ **x**: ratio of **global variables in the precision** (left = coarse abstraction)
 - ▶ **y**: **average explored abstract state space size**, relative to the last CEGAR iteration of the same task
- ✓ *Mann-Whitney U test confirms the significant advantage of abstraction-aware POR in the coarse quartiles*

Conclusion

Abstraction-aware POR is as good as the abstraction

The coarser the abstraction, the smaller D_{Π} , and the greater the reduction. If a property can be proven while ignoring many variables, the gain can be large.

- ▶ A **general** recipe: any POR algorithm $\times$ any abstract domain — necessary and sufficient correctness conditions, formulated with source sets and partial feasibility
- ▶ Side-effect: using D_S in the abstract state space is also sound as $D_{\Pi} \subseteq D_S$ (good news for those naively using POR in the abstract state space)
- ▶ When the abstraction is fine, the extension neither helps nor hurts much — however, AAPOR is more likely to find spurious counterexamples that can lead to more refinement rounds

Thank you!

Questions?